

A Unified Two-Dimensional Design Space for the MPI All-to-All Collective

Ke Fan¹[0000–0002–9563–6129], Guimu Guo²[0000–0002–3573–7446], and Sidharth Kumar³[0009–0007–0418–9962]

¹ Temple University, Philadelphia, PA, USA

² Rowan University, Glassboro, NJ, USA

³ University of Illinois Chicago, Chicago, IL, USA

ke.fan@temple.edu, guog@rowan.edu, sidharth@uic.edu

Abstract. The all-to-all collective sits on the critical path of many HPC and ML workloads, and its performance is highly sensitive to the process count P and the message size N . Prior work has made two parameters tunable, but never both in the same algorithm. The *radix* r is tuned in blocking round-by-round schedules, trading the number of rounds against the exchanged volume. The *batch-size* b , in contrast, is tuned in non-blocking batched schedules, trading network contention against concurrency. We find that the two can coexist in a single schedule, once these structurally divergent paradigms are reconciled. We do so by observing that the true dependency holds only across phases, leaving same-phase steps free to batch. On this basis we present GATA (Generalized All-To-All), a uniform all-to-all algorithm that tunes r and b jointly over a dense two-dimensional design space. Widely-deployed algorithms occupy only the boundary of this space, while its interior exposes configurations none of them can express. Selecting an all-to-all algorithm therefore becomes a matter of tuning two parameters within a single implementation. On Polaris at up to 8192 processes, the slowest and fastest (r, b) configurations at a fixed (P, N) differ by as much as $683\times$. Once tuned, GATA outperforms the strongest baseline by up to $3.4\times$ and the vendor-optimized `MPI_Alltoall` by up to $10.6\times$.

Keywords: Collectives · alltoall · tuning · parameterized algorithms

1 Introduction

Efficient data movement remains a defining challenge of exascale computing [3]. At the heart of this challenge lies collective communication, in which nearly every process participates in a single coordinated exchange. Because of this global scope, its cost scales with both the process count and the per-process data volume, making collectives difficult to scale efficiently [31]. In communication-intensive applications, collectives routinely consume over 40% of total runtime [9, 19], rising to 80% in large-scale Mixture-of-Experts (MoE) training [20]. Among collectives, all-to-all communication is the most bandwidth-intensive [29]. In the

all-to-all collective [24], every process sends a distinct block to every other process, so a single call on P processes transfers $P(P-1)$ blocks and saturates the bisection bandwidth of the interconnect. The standard defines two variants. In the *uniform* variant every process pair exchanges same-sized blocks, while in the *non-uniform* variant per-pair block sizes may vary. The collective lies on the critical path of a wide range of HPC and ML workloads, including FFT-based solvers [25], parallel relational algebra [23], graph analytics [5], and the expert-routing shuffles in MoE training [8]. This work targets the uniform variant.

Limitations of state-of-the-art approaches. All-to-all performance is highly sensitive to the process count P , the per-peer block size N , and the characteristics of the interconnect. Production MPI libraries such as MPICH [16] and Open MPI [4] address this sensitivity through algorithm switching, in which they ship several algorithms and select one at runtime based on static heuristics over (P, N) [2]. These algorithms fall into two families. Linear-time variants, such as the scattered algorithm [29], deliver every block directly to its destination in a single hop, completing in $\Theta(P)$ rounds and targeting large messages. Logarithmic-time variants based on the Bruck algorithm [7] instead route each block through several intermediate processes, completing in $\Theta(\log_2 P)$ rounds and targeting small-to-medium messages. Both families fix the *radix* r (log-base) at an extreme, $r=2$ for log-time and $r=P$ for linear-time.

Recent work [12, 10] demonstrates that treating radix r as a tunable parameter over $[2, P]$ outperforms either extreme across a wide range of (P, N) . Independently, scattered exposes a tunable *batch-size* b that partitions send/receive requests into smaller batches, mitigating network contention at scale. Despite this progress, the state of the art remains a collection of separate algorithms, tunable-radix variants included, and one must still be selected as (P, N) varies. Each algorithm is itself narrowly parameterized. Most expose no tuning knob, while a few allow tuning along a single axis, either the radix or the batch-size, but never both jointly. These limitations surface in practice. First, runtime switching among algorithms is brittle, failing for applications with dynamic communication behavior whose (P, N) shifts during execution. Second, even once a suitable algorithm is selected, it occupies only a point or a 1-D axis of the (r, b) space, leaving performance gains that no algorithm switch can recover. A unified design is therefore required, one that opens the (r, b) interior to joint tuning.

Key insights and contributions. We present GATA (Generalized All-To-All), a parameterized algorithm that generalizes existing uniform all-to-all algorithms into one design controlled by two jointly tunable parameters, the *radix* r and the *batch-size* b . The two govern distinct axes of communication cost, with r trading the number of rounds against the volume forwarded and b trading network contention against concurrency. Existing algorithms appear as special cases of the (r, b) grid (e.g., Bruck at $r=2, b=1$, and scattered [29] along $r=P$), while its interior holds faster configurations that none of them can express. Reaching this interior is not mechanical, because the two parameters belong to structurally divergent paradigms. The radix r is tuned in *blocking* round-by-round schedules (as in Bruck), while the batch-size b is tuned in *non-blocking* batched schedules

(as in scattered). A blocking schedule leaves no concurrency for b to dial. GATA reconciles the two paradigms by observing that the true dependency holds only across the schedule’s $\lceil \log_r P \rceil$ phases, which frees the $r-1$ steps within a phase to issue as one non-blocking batch. To our knowledge, this is the first design to expose joint (r, b) tuning for all-to-all. Conceptually, it also recasts collective tuning from a categorical selection among algorithms into a search over a structured two-dimensional (r, b) grid, opening it to standard numerical and learning-based methods. In summary, our paper makes the following contributions:

1. We propose GATA, a generalized parameterized algorithm for uniform all-to-all. GATA realizes a dense two-dimensional (r, b) design space, subsuming existing algorithms as special cases and reaching its high-performance interior without per-case branching.
2. We characterize the empirical (r, b) landscape and identify two distinct regimes, latency-bound and bandwidth-bound, separated by a sharp crossover, motivating a reference (r, b) selection policy driven by message size.
3. We perform a comprehensive evaluation on the Polaris supercomputer, scaling up to 8K processes. Across all (P, N) configurations, GATA outperforms or matches every baseline tested, achieving up to $3.4\times$ over the strongest baseline. Against the vendor-optimized implementation, GATA further delivers up to $10.6\times$ speedup.

2 Background

This section briefly reviews the classical all-to-all algorithms referenced throughout the paper to establish a clear context for our work.

2.1 Problem Formulation

With P processes ranked 0 to $P-1$, the all-to-all collective is defined as follows. Every process maintains a *send buffer* (initialized with data) and a *receive buffer* (initially empty), each logically partitioned into P data-blocks $S[0 \dots P-1]$ and $R[0 \dots P-1]$. During the collective, the process with rank p transmits its block $S[i]$ to the process with rank i and stores the block received from process i into its $R[i]$, for all $0 \leq i \leq P-1$. In the uniform variant, every data-block has the same fixed size N in bytes, so both buffers are contiguous 1-D arrays of size $P \cdot N$ bytes with blocks laid out in increasing rank order.

2.2 Existing All-to-all Algorithms

All-to-all algorithms fall into two families, distinguished by how they trade off the number of communication rounds against per-round data volume.

Linear-time algorithms. Linear-time variants complete in $\Theta(P)$ rounds and deliver each data-block directly to its destination in a single hop, with no intermediate buffering. They differ only in how the $P-1$ exchanges are scheduled. *Pairwise* uses blocking `MPI_Sendrecv` with an XOR-additive partner schedule ($p \leftrightarrow p \oplus i$ in round i), which is self-inverse and idle-free for power-of-two P . *Spread-out* [22] posts all $P-1$ non-blocking sends/receives concurrently under a round-robin schedule ($p \leftrightarrow (p+i) \bmod P$) that staggers traffic to avoid endpoint contention. *Basic Linear* (Open MPI) replaces round-robin with ascending-rank ordering, which is simpler but prone to contention on low-rank peers early in the schedule. *Scattered* [29] extends Spread-out with a tunable *batch-size* b , posting requests in batches to trade request-level parallelism against network contention.

Log-time algorithms. Bruck [7] is the classic logarithmic all-to-all, cutting the round count to $\lceil \log_2 P \rceil$ via *store-and-forward* routing. The algorithm proceeds in three stages. First, each process cyclically shifts its send buffer by its rank. Second, it performs $\lceil \log_2 P \rceil$ communication rounds, exchanging in round x at most $\lceil P/2 \rceil$ blocks with the partner at distance 2^x . Because some destinations require multiple hops, received blocks are placed back into the send buffer and forwarded in subsequent rounds. Third, an inverse rotation restores the destination order. The trade is fewer rounds for more forwarded volume, favoring small messages. Recent work [12, 10] generalizes Bruck to a tunable radix $r \in [2, P]$, which cuts the number of *phases* to $\lceil \log_r P \rceil$ but issues up to $r-1$ exchanges within each phase. Raising r therefore adds rounds while reducing forwarded volume. The heuristic $r \approx \sqrt{P}$ balances the two and is reported as optimal across a wide range of (P, N) .

2.3 Communication Cost Model

All-to-all collectives are typically realized by a sequence of point-to-point exchange rounds, where each process exchanges one or more data-blocks with its partners per round. Under the topology-agnostic Hockney model [17], the cost of a point-to-point communication is $T_{p2p} = \alpha + m \cdot \beta$, where α is the per-message startup latency and β is the per-byte bandwidth cost. The total cost of an all-to-all is therefore determined by two algorithm-dependent quantities, the number of rounds K and the total number of data-blocks D transmitted per process. Concretely, $T_{a2a} = \alpha \cdot K + \beta \cdot D \cdot N$, where N is the per-block size in bytes. K and D are inversely correlated, because a schedule with fewer rounds cannot reach every destination directly and must forward blocks through intermediates, which multiplies the volume. Hence K is the latency-related factor, dominant at small messages, while D is the bandwidth-related factor, dominant at large messages. This explains why logarithmic-time algorithms (low K) favor small messages, while linear-time algorithms (low D) favor large messages.

3 GATA: Generalized Uniform All-to-All Design

In this section, we present GATA, our generalized parameterized algorithm for uniform all-to-all. The key idea behind the design is to expose a *dense two-*

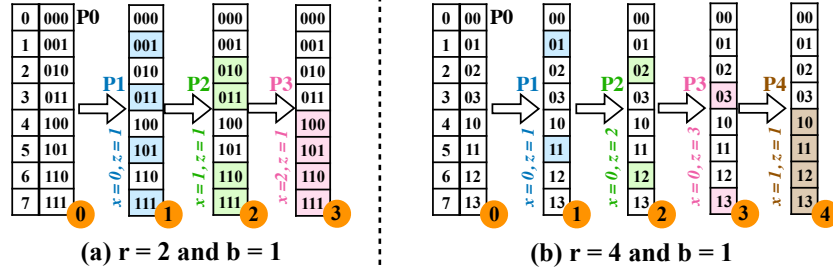


Fig. 1: An example of GATA on 8 processes (showing process P_0), demonstrating how the algorithm changes with varying radix, (a) $r=2$ and (b) $r=4$. We display only the $w = \lceil \log_r P \rceil$ phases. The data-block indices are first re-encoded in base r (⓪). Then, in each phase, P_0 sends all data-blocks whose x -th digit equals z (with $1 \leq z < r$) to the process at distance $z \cdot r^x$ (Ⓜ, Ⓝ, ...). Data-blocks sent in each phase are highlighted in color.

dimensional design space in which multiple existing all-to-all implementations (both log-time and linear-time variants) appear as isolated boundary points. Its interior contains configurations unreachable by any single one. Because r and b range over $[2, P]$ and $[1, r-1]$ respectively, the space is a grid of integer points rather than a true continuum, but it is dense enough that tuning traverses it in fine increments instead of jumping between a handful of predefined algorithms. This removes the brittleness of algorithm switching and enables fine-grained tuning per (P, N) configuration. The design space is realized through two tunable parameters. The radix r controls the total number of communication requests per process, and the batch-size b controls the maximum number of concurrent requests in the network. Jointly tuning them unlocks performance regimes beyond the reach of any single-parameter strategy.

Realizing this 2-D design is not automatic, because the blocking primitives of log-time implementations leave no concurrency for b to control, confining prior work [32, 12, 10] to tuning r alone. We formalize the parameter semantics in Section 3.1, present the implementation in Section 3.2, and analyze complexity in Section 3.3.

3.1 Parameter Semantics

Tunable radix implies blocking communication. GATA structures the all-to-all over P processes as a schedule parameterized by a radix r . The schedule executes $w = \lceil \log_r P \rceil$ phases, each containing up to $r-1$ point-to-point z -steps (one per nonzero base- r digit value). Every process encodes its P data-blocks in base r , yielding w digits per block in $\{0, \dots, r-1\}$ (e.g., step ⓪ of Figure 1(a) converts data-block indices to binary encoding at $r=2$ for $P=8$, giving $w=3$ digit positions and a single $z=1$ per position). Within digit position $x \in [0, w)$, each z -step ($z \in [1, r)$) sends to the partner at rank distance $z \cdot r^x$ those blocks whose

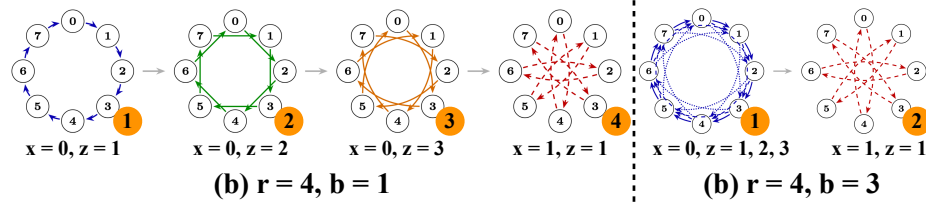


Fig. 2: An example of GATA with 8 processes, demonstrating how the algorithm changes with varying batch-size, (a) $b=1$ and (b) $b=3$. Each panel (①, ②, ...) is exactly one batched exchange, and arrows between panels indicate inter-phase synchronization. (a) At $b=1$, every z -step is its own batch, giving fully blocking communication. (b) With $b=r-1=3$, the three z -steps at each x collapse into a single non-blocking batch.

x -th digit equals z . For instance, Figure 1(b) shows this schedule for $P=8$ at $r=4$, where $x=0$ runs three z -steps ($z=1, 2, 3$) while $x=1$ runs only $z=1$ due to the top-digit deficit. There are $K = w(r-1) - d$ rounds (one per (x, z) pair) in total, where d is a slack term when P is not a power of r [12]. Each round transmits up to r^{w-1} blocks per process, totaling $D \leq w(r-1) \cdot r^{w-1}$ transmissions. At $r=2$ this gives $K=3$ and $D=12$, and at $r=4$ it gives $K=4$ and $D=10$. Increasing the radix therefore increases K while decreasing D .

Since D far exceeds the $P-1$ blocks each process ultimately delivers (e.g., 12 vs. 7 at $r=2$), blocks received in earlier rounds must be stored and re-transmitted in later ones. The schedule is therefore *round-synchronous*, because round $k+1$ reads slots round k has just written, so rounds cannot be safely interleaved. Almost all existing Bruck-style implementations [7, 30, 32, 10–12] accept this dependency and adopt blocking primitives, leaving r as the sole tuning parameter and foreclosing any latent concurrency the schedule might admit.

Tunable batch-size implies non-blocking communication. The batch-size b is the maximum number of in-flight exchange requests. It exposes a latency-versus-contention trade-off, where larger b hides per-request latency while smaller b relieves network contention. This trade-off has long been exploited in linear-time variants such as the scattered algorithm [29], which is inherently non-blocking and naturally tunes b to mitigate contention at large P . But in a strictly blocking schedule the axis has no analogue, since at $b=1$ every request completes before the next is posted, leaving no concurrency to dial. Figure 2 contrasts the extremes at $r=4$, where $b=1$ serializes each z -step at a given x (a) into a separate exchange, while $b=r-1=3$ (b) batches them into a single non-blocking exchange. For $b > 1$ to be correct, the concurrent z -steps within a batch, of which there are at most b , must read from disjoint byte ranges of the send-side staging area. GATA assigns each in-flight z a private offset, an invariant easily overlooked when batched extensions are bolted onto a single- z Bruck baseline.

Joint (r, b) tuning reconciles blocking and non-blocking communication. Together, r and b span a dense two-dimensional design space, which raises the key question this work addresses. *How can a log-time, round-synchronous algorithm admit a non-blocking concurrency axis without breaking its phase struc-*

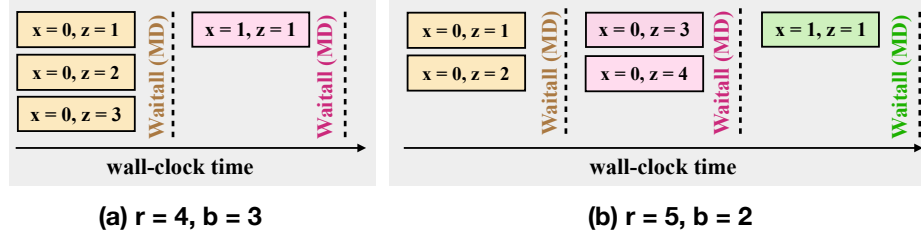


Fig. 3: The (r, b) design space of GATA with 8 processes. Each block, labeled by its (x, z) pair, represents a single communication round. Blocks stacked vertically are issued together within one batch as non-blocking requests. Blocks arranged horizontally are separate batches, each terminated by an MPI_WAITALL and thus blocking against the next. Tuning (r, b) interpolates between the blocking and non-blocking regimes in fine increments.

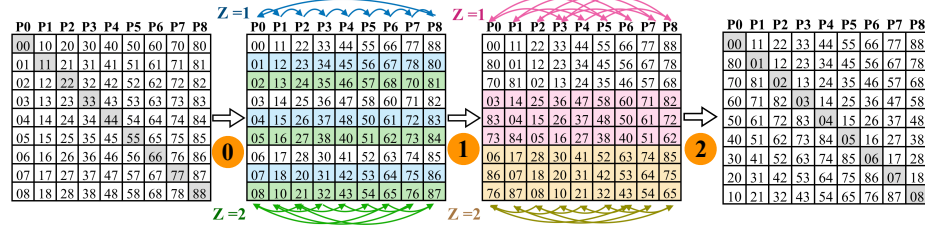


Fig. 4: Overview of GATA with $P = 8$, $r = 3$, and $b = 2$ (log-time style). ① depicts the send buffer before and after the initial rotation. ① and ② depict the data before and after each of the two communication rounds. Within each round, the two z -steps (colored differently) are sent together as a single non-blocking batch. The rightmost panel shows that every process has received all required data, but still needs a final inverse rotation to restore the correct order.

ture? Our solution rests on a structural observation. The round-synchronicity observed in existing implementations is overly conservative, because the true dependency holds only between phases (x), not between z -steps within the same phase. The $r-1$ z -steps sharing a fixed x access disjoint slots and can run concurrently. GATA therefore posts as many as b same-phase z -steps concurrently as non-blocking requests, retaining a blocking boundary across phases to preserve the inter-phase dependency. For example, Figure 3 gives two (r, b) examples at $P=8$. In (a), $r=4, b=3$ collapses each phase's three z -steps into a single non-blocking batch, the $b=r-1$ boundary. In (b), $r=5, b=2$ splits each phase's four z -steps into two two-way non-blocking batches, an interior point. The resulting schedule combines the logarithmic phase structure of Bruck with the latency hiding of the asynchronous scattered algorithm within one (r, b) -parameterized algorithm. Section 3.3 formalizes this decomposition under the α - β cost model.

Algorithm 1 GATA($r, b, \text{sendbuf}, N, \text{recvbuf}, \text{comm}$): uniform all-to-all.

```

1:  $w \leftarrow \lceil \log_r P \rceil$ ;  $d \leftarrow \text{top-digit deficit}$ 
2: if  $r = P$  then ▷ linear-time fast path: skip rotations and staging
3:    $\text{recvbuf}[R] \leftarrow \text{sendbuf}[R]$ 
4:   for  $k = 1, 1+b, \dots, P-1$  do
5:      $b' \leftarrow \min(b, P-k)$  ▷ tighten last batch
6:     post MPI_Irecv/MPI_Isend for  $s \in [0, b')$  between  $\text{sendbuf}$  and
        $\text{recvbuf}$ , peers  $(R \pm (k+s)) \bmod P$ 
7:     MPI_Waitall
8:   end for
9:   return
10: end if
11: allocate scratch  $\text{tmp}$ ,  $\text{tsend}$ ,  $\text{trecv}$  (cached across calls)
12:  $\text{tmp}[i] \leftarrow \text{sendbuf}[(i+R) \bmod P]$  for  $i = 0, \dots, P-1$  ▷ pre-rotation
13: for  $x = 0, \dots, w-1$  do ▷ phase  $x$ : partner distance  $r^x$ 
14:    $z_e \leftarrow (x = w-1) ? r-d : r$  ▷ tighten last phase to surviving digits
15:   for  $k = 1, 1+b, \dots, z_e-1$  do ▷ batched  $z$ -loop
16:      $b' \leftarrow \min(b, z_e-k)$  ▷ tighten last batch
17:     pack: for  $s \in [0, b')$ , copy  $\text{tmp}$ -blocks with digit- $x = k+s$  into  $\text{tsend}$ 
18:     post: MPI_Irecv/MPI_Isend for  $s \in [0, b')$ , peers  $(R \pm (k+s)r^x) \bmod P$ 
19:     MPI_Waitall
20:     unpack: scatter  $\text{trecv}$  back into  $\text{tmp}$ 
21:   end for
22: end for
23:  $\text{recvbuf}[(R-i+P) \bmod P] \leftarrow \text{tmp}[i]$  for  $i = 0, \dots, P-1$  ▷ post-rotation

```

3.2 Algorithm Implementation

GATA preserves the standard MPI all-to-all interface, exposing r and b as the only additional arguments, so existing applications can adopt it as a drop-in replacement. Algorithm 1 presents the pseudocode of GATA. To maximize efficiency, the implementation embeds a runtime fast path (line 2) that smoothly switches between Bruck-style log-time and scattered-style linear-time regimes without any algorithm-selection branch. When (r, b) selects the linear-time regime ($r=P$), GATA skips the rotation and staging overhead that logarithmic schedules require. Figure 4 illustrates the log-time form at $P=8, r=3, b=2$, with three stages, an initial pre-rotation (line 12), a 2-round batched communication (lines 13–20), and a final inverse rotation (line 23). This section also discusses the implementation considerations the (r, b) design introduces and characterizes how GATA specializes to existing algorithms at the grid boundaries.

Persistent Scratch Slab. The store-and-forward schedule must retain received blocks for later re-transmission, requiring a persistent staging buffer. Moreover, whenever $r < P-1$ each round exchanges multiple non-contiguous blocks per peer, which must be *packed* (line 17) into a contiguous send region before transmission and *unpacked* (line 20) back into their slots upon receipt. MPI implementations [1, 4] conventionally use derived datatypes, but prior work [11]

has shown that explicit `memcpy` outperforms datatype-based packing, especially for small messages. GATA therefore needs three buffers, namely `tmp` for store-and-forward, plus `tsend` and `treceive` for manual pack/unpack. Because raising b collapses as many as b of the z -steps into a single non-blocking batch, `tsend` and `treceive` must hold an entire batch’s data and are correspondingly larger than in a single- z schedule. The aggregate footprint reaches up to $3PN$ bytes and is provisioned only on the log-time path (line 11). `malloc/free`-ing three independent allocations per call makes allocator overhead rival the entire collective in the small- N regime. This effect is amplified by iterative workloads that invoke the collective multiple times with varying (P, N) , such as fixed-point graph algorithms [15, 26] and MoE expert routing [20]. GATA optimizes this by fusing the three buffers into a single slab allocated lazily, grown on demand and never freed, so the operating system reclaims it at process exit. This eliminates allocator overhead after the first call, and the slab is the only state GATA retains across invocations.

Corner-case consideration. A unified (r, b) -parameterized schedule must remain correct at two irregular corners of the (r, b) grid. The *digit-deficit residual* arises when P is not a power of r . In this case, the final phase carries fewer than $r-1$ z -values. The *batching residual* arises when b does not divide $r-1$. In this case, each phase ends with a partial tail batch of $(r-1) \bmod b$ requests. A naive implementation resolves these with per-corner branching. GATA instead handles both by tightening the loop bounds at runtime. On the final outer iteration (line 13), the inner z -loop iterates only over the surviving digit values rather than the full $r-1$ range, eliminating the digit-deficit residual. Within each phase, the last batch (line 15) is sized to the number of z -steps still remaining rather than to a fixed b , eliminating the batching residual. Thus, the four-way corner-case matrix collapses into one code path with no branching.

Boundary cases. The (r, b) grid extremes specialize GATA to existing algorithms. For $r < P$, $b=1$ recovers radix- r Bruck [7], and at $r=2$ this is classic Bruck. Setting $r=P$ collapses both rotations away into a single direct-exchange phase and matches MPICH’s scattered all-to-all [29] with `block=b`. This spans from classical pairwise exchange at $b=1$ to the fully concurrent linear schedules (basic linear and spread-out [22]) at $b=P-1$, where each batch holds up to b send/rcv pairs. Setting $r=P-1$ yields essentially the same behavior as $r=P$, since neither requires forwarding. Interior (r, b) pairs lie between these endpoints, opening configurations no existing algorithm reaches.

3.3 Algorithm Analysis

We analyze GATA’s communication time under the Hockney model [17] (α - β). GATA executes $w = \lceil \log_r P \rceil$ phases, each issuing $r-1$ z -steps in batches of b , so each phase incurs $\lceil (r-1)/b \rceil$ synchronizations. Summing one α per batched wait gives the latency cost

$$T_\alpha(r, b, P) = \alpha \cdot \lceil \log_r P \rceil \cdot \lceil (r-1)/b \rceil, \quad (1)$$

whose two factors separate the roles of the parameters. The phase count $\lceil \log_r P \rceil$ depends on r alone, while the number of batched waits per phase, $\lceil (r-1)/b \rceil$, depends on both, since the $r-1$ z -steps a phase must issue are themselves set by r . The latency cost therefore couples the two parameters, and the coupling is also structural, because b is meaningful only over the range $[1, r-1]$ that r defines. Neither parameter can be tuned to its optimum in isolation.

In each phase, the $\frac{r-1}{r}$ fraction of blocks with a non-zero digit at the active position are transmitted, giving the bandwidth bound

$$T_\beta(r, P, N) \leq \beta \cdot N \cdot P \cdot \frac{r-1}{r} \cdot \lceil \log_r P \rceil, \quad (2)$$

independent of b , since batching redistributes the same total volume across fewer synchronizations. The total cost $T = T_\alpha + T_\beta$ omits a $O(\gamma NP \log_r P)$ pack/unpack term that is dominated by T_β on typical interconnects ($\beta \gg \gamma$). Both axes raise network contention, which must be tuned empirically.

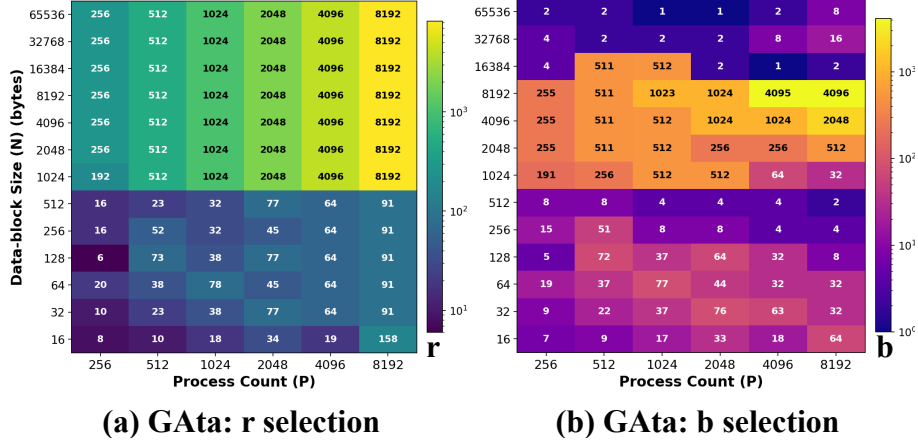
4 Evaluation

We evaluate GATA through micro-benchmarks on the Polaris supercomputer at Argonne National Laboratory, using Cray MPICH 8.1.16. Polaris comprises 560 nodes, each equipped with a 32-core AMD CPU and four NVIDIA A100 GPUs, for an aggregate peak performance of 44 petaflop/s, and its nodes are interconnected by a Slingshot-based Dragonfly topology. All experiments launch 32 MPI processes per node, one per CPU core.

Our evaluation pursues two objectives. First, to characterize the performance landscape of the 2-D parameter space (r, b) , we conduct sweeps that span a broad range of configurations and expose how performance varies with parameter choice (Section 4.1). Second, we benchmark GATA at its best (r, b) configuration against our re-implementations of the uniform all-to-all algorithms presented in Section 2 and the vendor-optimized `MPI_Alltoall` (Section 4.2). Across all experiments, we vary the process count P , the per-peer block size N , the radix r , and the batch-size b . Each configuration is repeated for at least 20 iterations, and we report the median with the standard deviation (shown as error bars).

4.1 Performance Analysis of GATA

To understand the performance landscape of GATA, we conducted an extensive sweep that varied P from 256 to 8192, N from 16 to 65536 bytes, r from 2 to P , and b from 1 to $r-1$, yielding tens of thousands of (P, N, r, b) configurations in total. Figures 5(a) and (b) summarize these results as heatmaps. Both figures share the same $6 \times 13 = 78$ grid of (P, N) cells, where the x-axis is the process count P and the y-axis is the per-peer block size N (bytes). Each cell reports the optimal r or b at that (P, N) pair. Figure (a) is annotated and colored by the optimal radix r , Figure (b) by the corresponding batch-size b . The color bar

Fig. 5: GATA tuning, (a) radix r and (b) batch-size b selection.

on the right maps cell color to the numeric value, with brighter colors indicating larger optimal r or b . The two heatmaps together show the optimal (r, b) combination for every (P, N) pair.

From the experimental results, we observe that the choice of (r, b) matters by an order of magnitude almost everywhere. Across all (P, N) pairs, the worst-to-best ratio within a sweep can reach up to $683\times$. For example, at $P = 8192$, $N = 2$ the best configuration ($r = 158, b = 64$) runs in $652 \mu s$ while the worst takes 446 ms. Even in the most forgiving pair ($P = 2048$, $N = 128$), the worst is still $4.7\times$ slower than the best. The (r, b) pair is therefore not a fine-tuning knob but a make-or-break choice.

The two heatmaps further reveal two clearly separated regimes, joined by a sharp transition. The lower half of each heatmap is the *latency-bound* region (small N). Here the optimum sits near $r \approx \sqrt{P}$, with b close to the full-batch limit $r-1$, the classical Bruck point. For example, $P = 4096$, $N = 4$ selects ($r = 64, b = 63$). As N grows within this regime, b retracts, and at $P = 8192$, $N = 64$ the optimum is already ($r = 91, b = 2$). The top rows form the *bandwidth-bound* region (large N). Here the optimum jumps to $r = P$ with b as large as the network’s concurrent-send budget permits. For example, $P = 4096$, $N = 1024$ selects ($r = 4096, b = 4095$), and $P = 8192$, $N = 512$ selects ($r = 8192, b = 2048$). These are the pairwise/linear corners of the (r, b) grid, where the forwarding penalty $\eta(r, P)$ collapses to one and GATA degenerates into a single round of batched direct exchanges. Concretely, at both $P = 4096$ and $P = 8192$, the optimal radix jumps directly from $\sqrt{P}$ at $N = 64$ to P at $N = 128$, so a single doubling of N switches GATA from a multi-round Bruck pattern to a single-round direct exchange. This coincides with the α - β crossover, where the block size is large enough for the bandwidth cost to dominate the latency.

We also observe several findings that depart from traditional expectations. The classical $r = \sqrt{P}$ heuristic [12, 10] describes only the latency-bound edge. Across all the results, in only 28% of cells does the optimal r fall within $\pm 30\%$

of $\sqrt{P}$, while in 53% the optimum is $r = P$ outright. The optimum does not always use the maximum batch size. In 63% of the cells the best b is strictly smaller than $r-1$, sometimes by a wide margin. At $P = 8192$, $N = 8192$, for instance, the optimum is $(r = 8192, b = 8)$, splitting a single round into 1024 sub-batches. This is likely a hardware limit, where the network can only handle a bounded number of concurrent sends, so issuing all $P-1$ sends together causes contention, while smaller batches stay within that limit.

Taken together, these observations point to a practical recipe for selecting (r, b) from N . For N up to about 32 bytes, picking r near $\sqrt{P}$ and setting $b = r-1$ is sufficient. For N of 256 bytes or more, $r = P$ with b at most a few thousand is the right choice. The narrow band of 64–128 bytes requires a small search over a handful of (r, b) values.

4.2 Performance Comparison

To demonstrate the efficiency of GATA, this section compares it against six uniform all-to-all baselines spanning the vendor implementation, four widely-deployed classical algorithms, and a strong radix-tuned reference (see Section 2). The goal is twofold. We show that the (r, b) parameterization subsumes most existing all-to-all implementations as special cases of its grid and, once tuned, matches or outperforms each of them, and we show that jointly tuning the two-dimensional (r, b) grid is strictly better than tuning either axis alone.

Figure 6 plots the comparison results of GATA. The baselines fall into three groups, the vendor `MPI_Alltoall` from Cray MPICH, four widely-deployed all-to-all algorithms, and a Tuned Bruck baseline with $r = \sqrt{P}$ whose efficiency has been demonstrated in prior work [12, 10]. From the results, we observe that against the re-implemented baselines, GATA outperforms or at least matches all of them across all cases, even the strongest baseline, with a peak speedup of $3.4\times$. In addition, against the vendor `MPI_Alltoall`, GATA also wins on the vast majority of configurations, by up to $10.6\times$. On a small fraction of configurations the vendor edges slightly ahead, which we attribute to its proprietary hardware-level optimizations rather than to a deficiency of the (r, b) design.

To make the comparison concrete, we now zoom in on three regimes that exercise different parts of the (r, b) grid. The *latency-bound corner* (small N , large P) is where the per-round message-startup cost dominates, the *bandwidth-bound corner* (large N) is where the forwarding penalty $\eta(r, P)$ governs the cost, and the *medium-message transition band* ($256 < N < 1024$ bytes) is where the two effects overlap. These bins are coarser than the sharp optimal- r jump seen in the heatmap at $N \in [64, 128]$ bytes, chosen so that each bin contains enough (P, N) configurations to characterize baseline behavior. For each regime we report how GATA compares against the vendor and the baselines, and we attribute the gap to the specific axis of the (r, b) grid that GATA exploits.

Latency-bound region (small N , large P). The dominant cost in this corner is per-round latency. The vendor falls back to a basic-linear schedule at the smallest sizes, so at $P = 1024$, $N = 2$, GATA runs in $114\ \mu\text{s}$ against the vendor’s $1.21\ \text{ms}$ ($10.6\times$). Classic Bruck competes by virtue of its $\log_2 P$ scaling,

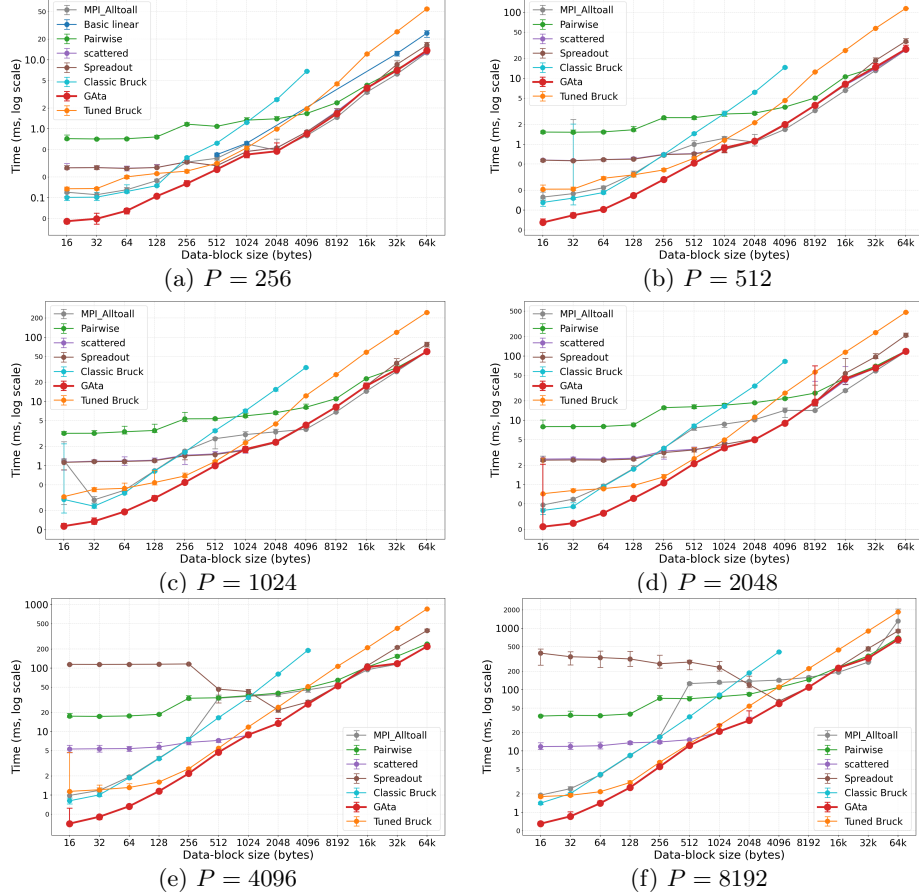


Fig. 6: GATA vs. widely-used baselines and vendor MPI_Alltoall on Polaris under varying P . Tunable algorithms (scattered and GATA) report their best configuration per (P, N) , and Tuned Bruck uses its reported-optimum $r = \sqrt{P}$.

an exponential improvement over the vendor’s linear schedule, but still loses $1.4\text{--}3.3\times$ to GATA, because GATA raises the radix to $\sqrt{P}$ and batches the resulting z -steps into 1–2 effective `Waitall` boundaries. Tuned Bruck then isolates the batching contribution. In the small- N region ($N \leq 256$ bytes) GATA’s speedup over Tuned Bruck is $3.26\times$. Both run radix- $\sqrt{P}$ Bruck and differ only in whether the $r-1$ z -steps issue as a single non-blocking batch or sequentially, so this separation is a direct measurement of the value the b knob adds to a classical Bruck schedule. Notably, Tuned Bruck itself beats every non-vendor user-space baseline on 24% of cells, so the speedup is measured against a strong baseline.

Bandwidth-bound region (large N). In this corner, the rankings reshuffle. The vendor closes most of its gap via a near-pairwise schedule and hardware collective offload, so GATA’s advantage drops below $3\times$, and on $\sim 30\%$ of large- N cells the vendor edges ahead by 5–20%. Moreover, the range of N over

which GATA beats the vendor widens as P grows, so GATA scales well to large process counts. Classic Bruck falls catastrophically, since its forwarding penalty $\eta(2, P) \approx \log_2 P$ inflates the bandwidth term by $12\times$ at $P = 4096$, and the $15.1\times$ max gap (219 ms vs 3.3 s at $P = 4096$, $N = 8192$) directly quantifies it. Tuned Bruck, with $\eta(\sqrt{P}, P) \approx 2$, remains the strongest non-vendor baseline in this regime. GATA still wins by $4.15\times$, but via a different mechanism, migrating r to P to drive the forwarding penalty to one, whereas Tuned Bruck stays pinned at $\sqrt{P}$ and pays the $2\times$ bandwidth penalty. At $P = 512$, $N = 8192$, GATA at $r = 512$ runs in 27.7 ms against Tuned Bruck’s 114.9 ms, a gap that comes entirely from the radix migration the (r, b) grid permits.

Medium-message transition band ($256 < N < 1024$ bytes). Here the two regimes overlap and GATA’s advantage over Tuned Bruck shrinks to $1.23\times$. This is the only region where Tuned Bruck stays within 25% of GATA on every configuration. The latency penalty of $b = 1$ has been amortized by the moderate message size, while the bandwidth penalty of $r = \sqrt{P}$ has not yet started to bite, so the classical $\sqrt{P}$ Bruck tuning happens to be near-optimal. The corollary for an auto-tuner is that a fixed-radix Bruck schedule suffices only inside this band, while the small- N and large- N corners both require the (r, b) grid to move away from $\sqrt{P}$, in opposite directions. Across those two corners, jointly tuning the 2-D (r, b) grid yields a further $\sim 2\times$ median speedup over Tuned Bruck, which tunes r alone, confirming that neither axis alone is sufficient.

5 Related Work

All-to-all is difficult to scale due to its quadratic per-call message volume, and prior optimization efforts have proceeded along independent lines. We organize the related work into four categories, namely tunable-radix collective designs, Bruck-schedule optimizations, locality-aware and hierarchical decompositions, and library-level algorithm selection.

Tunable-radix collective designs. Several efforts [31, 21, 10, 12, 28] treat the radix as a tuning axis rather than fixing it, though most target collectives other than all-to-all. Wilkins et al. [31] parameterized allreduce, broadcast, and allgather over varying radices and observed that the canonical choices $r=2$ and $r=P$ are rarely optimal, but they neither derived a selection rule nor addressed all-to-all. Jocksch et al. [21] swept radices for allgather, reduce-scatter, and allreduce under a fully-connected k -port model for FFTs, tying the radix to the port count ($r=k+1$). The radix was therefore tuned only by varying k rather than for fixed hardware. Taru et al. [28] demonstrated high-radix collectives but selected radices from installation-time measurements without an explicit cost model, advocating high radices for short messages and low radices for long ones. Gainaru et al. [14] derived a formula for the round count of Bruck-like algorithms when $\log_r P$ is non-integer, but focused on memory layouts rather than radix selection. Closest to our work, parameterized Bruck [12] and ParLogNa [10] sweep r over $[2, P]$ for all-to-all and propose a radix-selection heuristic, but they assume an implicit blocking schedule that leaves latent concurrency unexposed.

Bruck-schedule optimizations. Some work focuses on optimizing Bruck-style algorithms. Träff et al. [30] proposed two refinements to Bruck. Their *mod-*

ified Bruck avoids the final inverse rotation by reordering the initial rotation to anticipate the destination layout. Their *zero-copy Bruck* removes internal memory copies via a temporary staging buffer, demonstrating effectiveness only over a narrow message-size range. Xu et al. [32] replaced the initial rotation with an index array, deferring the physical reordering and improving performance when the bandwidth term D is small. Fan et al. [11] combined these ideas to eliminate both rotations and additionally showed that explicit `memcpy`-based packing outperforms MPI derived datatypes, particularly for small messages.

Locality-aware and hierarchical collectives. The cost asymmetry between intra- and inter-node communication motivates hierarchical decompositions of all-to-all. Jackson and Booth [18] and, independently, Plummer and Refson [27] aggregate intra-node sends before performing a sparse inter-node exchange. Bienz et al. [6] extended this to allgather with Bruck by grouping processes into low-overhead communication regions. Jocksch et al. [21] additionally exploited node-local shared memory by allocating shared buffer exchanges.

Library-level algorithm selection. Production MPI libraries select among a fixed set of all-to-all variants via static heuristics on (P, N) . For example, MPICH uses compile- and runtime thresholds [29], Open MPI [4] exposes MCA parameters for runtime overrides, and vendor stacks ship closed-source selectors. Faraj and Yuan [13] explored automated selection within a discrete-algorithm framework, choosing the single best algorithm per (P, N) from a fixed pool.

6 Conclusion

In this paper, we presented GATA, a parameterized design for uniform all-to-all that exposes a dense two-dimensional (r, b) design space, in which existing algorithms appear as boundary cases while the interior holds configurations none of them can reach. By jointly tuning the radix r and the batch-size b , GATA unifies blocking and non-blocking communication in one code path, recasting all-to-all tuning from a categorical choice among algorithms into a search over a structured grid. On Polaris up to 8192 processes, GATA matches or outperforms every baseline, by up to $3.4\times$ over the strongest and $10.6\times$ over the vendor. The optimum partitions cleanly into a *latency-bound* regime ($r \approx \sqrt{P}$, near-full batch) and a *bandwidth-bound* regime ($r = P$, sub-maximum batch), yet most optima sit at a radix away from $\sqrt{P}$ or a batch below $r-1$, outside the reach of any existing algorithm. These findings suggest that reformulating standard collectives as densely parameterized families, paired with a cost-model- or learning-based selector, is a productive direction for further work. The natural next step is the non-uniform `MPI_Alltoallv`, where per-pair block sizes make the packing and forwarding costs rank-dependent.

7 Acknowledgement

This work was funded in part by NSF PPoSS large grant CCF-2316157, NSF CRII-2245792, and NSF SHF Small grant CCF-2221811. We are thankful to the ALCF’s Director’s Discretionary (DD) program for providing us with compute hours to run our experiments on the Polaris supercomputer located at the Argonne National Laboratory.

References

1. MPICH home page. <https://www.mpich.org>
2. Beni, M.S., Cosenza, B., Hunold, S.: MPI collective algorithm selection in the presence of process arrival patterns. In: 2024 IEEE International Conference on Cluster Computing (CLUSTER). pp. 108–119. IEEE (2024)
3. Bernholdt, D.E., Boehm, S., Bosilca, G., Venkata, M.G., Grant, R.E., Naughton, T., Pritchard, H.P., Schulz, M., Vallee, G.R.: A survey of MPI usage in the US exascale computing project. *Concurrency and Computation: Practice and Experience* **32**(3), e4851 (2020)
4. Bernholdt, D.E., Bosilca, G., Bouteiller, A., Brightwell, R., Ciesko, J., Dosanjh, M.G.F., Georgakoudis, G., Laguna, I., Levy, S., Naughton, T., Olivier, S.L., Pritchard, H.P., Schonbein, W., Schuchart, J., Shehata, A.: Taking the MPI standard and the Open MPI library to exascale. *International Journal of High Performance Computing Applications* (2024). <https://doi.org/10.1177/10943420241265936>
5. Besta, M., Podstawski, M., Groner, L., Solomonik, E., Hoefer, T.: To push or to pull: On reducing communication and synchronization in graph computations. In: Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '17). pp. 93–104 (2017)
6. Bienz, A., Gautam, S., Kharel, A.: A locality-aware Bruck allgather. In: Proceedings of the 29th European MPI Users' Group Meeting. pp. 18–26 (2022)
7. Bruck, J., Ho, C.T., Kipnis, S., Upfal, E., Weathersby, D.: Efficient algorithms for all-to-all communications in multiport message-passing systems. vol. 8, pp. 1143–1156 (1997)
8. Chen, C.C., Khorassani, K.S., Anthony, Q.G., Shafi, A., Subramoni, H., Panda, D.K.: Highly efficient Alltoall and Alltoallv communication algorithms for GPU systems. In: 2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). pp. 24–33 (2022)
9. Chunduri, S., Parker, S., Balaji, P., Harms, K., Kumaran, K.: Characterization of MPI usage on a production supercomputer. In: SC '18: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis. pp. 386–400. IEEE Press (2018)
10. Fan, K., Domke, J., Ba, S., Kumar, S.: Parameterized algorithms for non-uniform all-to-all. In: Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '25). ACM (2025)
11. Fan, K., Gilray, T., Pascucci, V., Huang, X., Micinski, K., Kumar, S.: Optimizing the Bruck algorithm for non-uniform all-to-all communication. In: Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing. pp. 172–184 (2022)
12. Fan, K., Petruzza, S., Gilray, T., Kumar, S.: Configurable algorithms for all-to-all collectives. In: ISC High Performance 2024 Research Paper Proceedings (39th International Conference). pp. 1–12 (2024)
13. Faraj, A., Yuan, X.: Automatic generation and tuning of MPI collective communication routines. In: Proceedings of the 19th Annual International Conference on Supercomputing. pp. 393–402 (2005)
14. Gainaru, A., Graham, R.L., Polyakov, A., Shainer, G.: Using InfiniBand hardware gather-scatter capabilities to optimize MPI all-to-all. In: Proceedings of the 23rd European MPI Users' Meeting (2016)

15. Gilray, T., Kumar, S., Micinski, K.: Compiling data-parallel Datalog. In: Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction (CC 2021). pp. 23–35 (2021)
16. Guo, Y., Raffanetti, K., Zhou, H., Balaji, P., Si, M., Amer, A., Iwasaki, S., Seo, S., Congiu, G., Latham, R., Oden, L., Gillis, T., Zambre, R., Ouyang, K., Archer, C., Bland, W., Jose, J., Sur, S., Fujita, H., Durnov, D., Chuvelev, M., Zheng, G., Brooks, A., Thapaliya, S., Doodi, T., Garazan, M., Oyanagi, S., Snir, M., Thakur, R.: Preparing MPICH for exascale. *International Journal of High Performance Computing Applications* (2025). <https://doi.org/10.1177/10943420241311608>
17. Hockney, R.W.: The communication challenge for MPP: Intel Paragon and Meiko CS-2. *Parallel Computing* **20**(3), 389–398 (1994)
18. Jackson, A., Booth, S.: Planned AlltoAllv: a cluster approach. Tech. rep., EPCC (2004)
19. Jiang, C., Tian, Y., Jia, Z., Zheng, S., Wu, C., Wang, Y.: Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping. In: *Proceedings of Machine Learning and Systems (MLSys)* (2024)
20. Jin, C., Jiang, Z., Bai, Z., Zhong, Z., Liu, J., Li, X., Zheng, N., Wang, X., Xie, C., Huang, Q., Heng, W., Ma, Y., Bao, W., Zheng, S., Peng, Y., Lin, H., Liu, X., Jin, X., Liu, X.: MegaScale-MoE: Large-scale communication-efficient training of mixture-of-experts models in production. *arXiv preprint arXiv:2505.11432* (2025)
21. Jocksch, A., Ohana, N., Lanti, E., Karakasis, V., Villard, L.: Optimised allgather, reduce_scatter and allreduce communication in message-passing systems (2020)
22. Kang, Q., Ross, R., Latham, R., Lee, S., Agrawal, A., Choudhary, A., Liao, W.k.: Improving all-to-many personalized communication in two-phase I/O. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. pp. 1–13. IEEE (2020)
23. Kumar, S., Gilray, T.: Distributed relational algebra at scale. In: *International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE (2019)
24. Message Passing Interface Forum: MPI: A message-passing interface standard version 5.0. Tech. rep., Message Passing Interface Forum (2025), <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>
25. NVIDIA Corporation: Multinode multi-GPU: Using NVIDIA cuFFTMp FFTs at scale. <https://developer.nvidia.com/blog/multinode-multi-gpu-using-nvidia-cufftmp-ffts-at-scale/> (2022)
26. Patel, S., Dave, B., Kumbhani, S., Desai, M., Kumar, S., Chaudhury, B.: Scalable parallel algorithm for fast computation of transitive closure of graphs on shared memory architectures. In: *2021 IEEE/ACM 6th International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*. pp. 1–9 (2021)
27. Plummer, M., Refson, K.: An LPAR-customized MPI Alltoallv for the materials science code CASTEP. Tech. rep., EPCC (2004)
28. Taru, D., Islam, N., Zheng, G., Kalidas, R., Langer, A., Garzaran, M.: High radix collective algorithms. In: *Proceedings of EuroMPI 2021* (2021)
29. Thakur, R., Rabenseifner, R., Gropp, W.: Optimization of collective communication operations in MPICH. *International Journal of High Performance Computing Applications* **19**(1), 49–66 (2005)
30. Träff, J.L., Rougier, A., Hunold, S.: Implementing a classic: Zero-copy all-to-all communication with MPI datatypes. In: *Proceedings of the 28th ACM International Conference on Supercomputing* (2014)

31. Wilkins, M., Wang, H., Liu, P., Pham, B., Guo, Y., Thakur, R., Dinda, P., Har-
davellas, N.: Generalized collective algorithms for the exascale era. In: 2023 IEEE
International Conference on Cluster Computing (CLUSTER). pp. 60–71. IEEE
(2023)
32. Xu, C., Venkata, M.G., Graham, R.L., Wang, Y., Liu, Z., Yu, W.: SLOAVx: Scal-
able logarithmic alltoallv algorithm for hierarchical multicore systems. In: 2013
13th IEEE/ACM International Symposium on Cluster, Cloud, and Grid Comput-
ing. pp. 369–376. IEEE (2013)